

Lecture 13: DNN Interpretability: Internal

CS 580B1: Trustworthy Machine Learning

Fall 2024

Acknowledgments

Course slides derived from material created by Rajeev Alur and Osbert Bastani at UPenn ([link](#) to their course)

Other relevant courses on Trustworthy Machine Learning:

[CS 329T](#) at Stanford

[CS 521](#) at UIUC

[ECE1784H/CSC2559H](#) at U Toronto

Course logistics

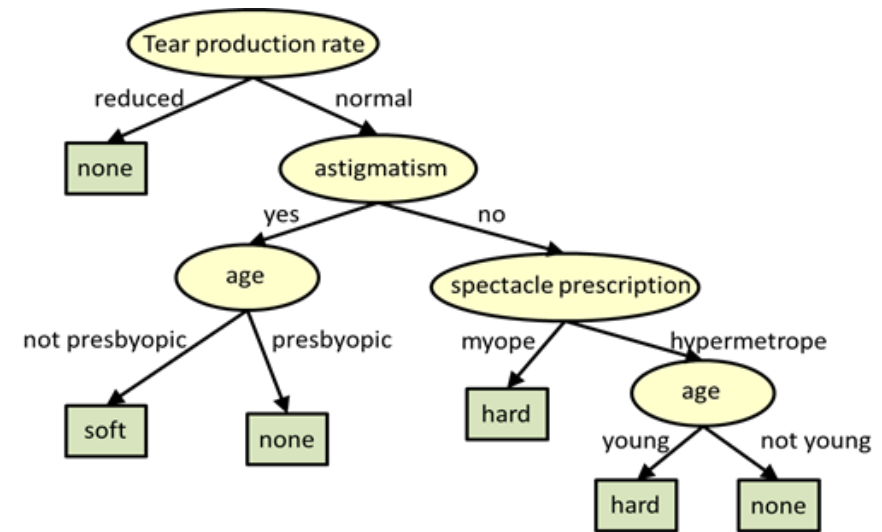
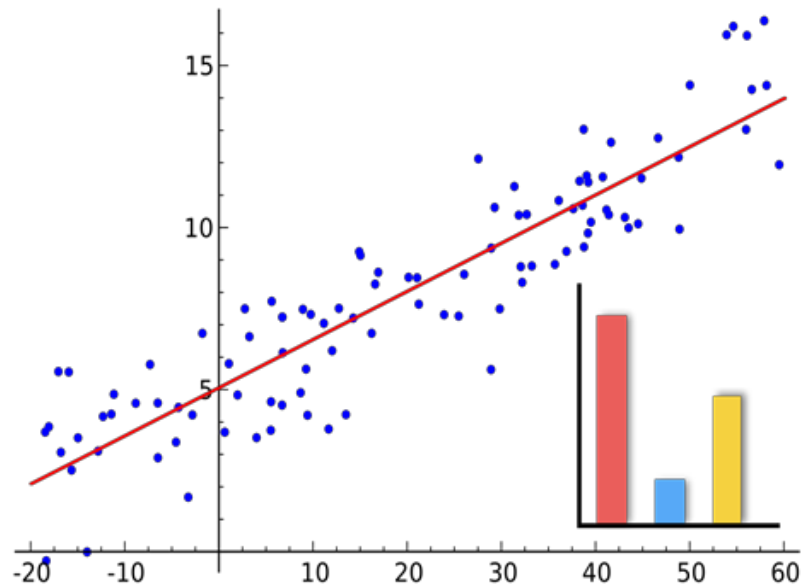
- Reviews:
 - Review due tomorrow at 11:59 PM
 - Paper to review: [Understanding intermediate layers using linear classifier probes](#) by Alain and Bengio
- Projects:
 - Form teams!
 - Enter team members here
- Non-standard schedule for the next 2 weeks

Types of interpretability techniques

Inherent

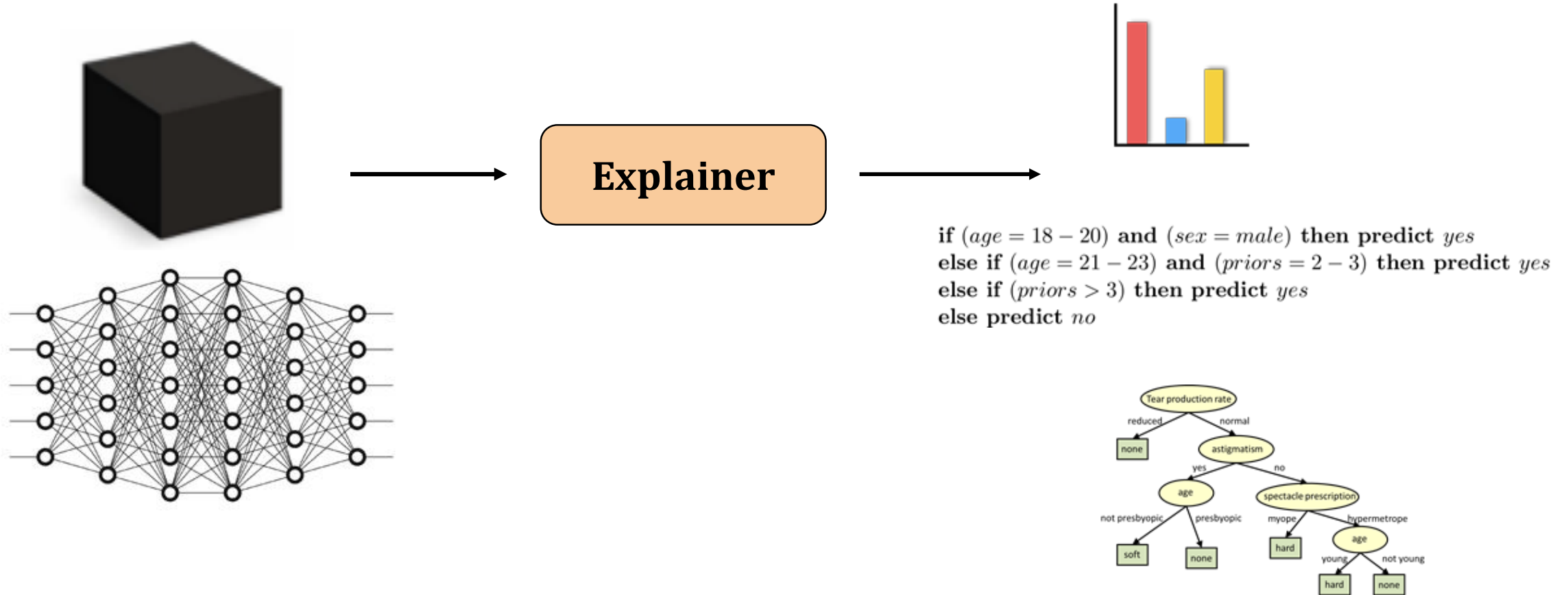
Post-hoc

Inherently interpretable models



if (*age* = 18 – 20) and (*sex* = male) then predict *yes*
else if (*age* = 21 – 23) and (*priors* = 2 – 3) then predict *yes*
else if (*priors* > 3) then predict *yes*
else predict *no*

Post-hoc interpretability



Types of interpretability techniques

Inherent

- Linear models
- Decision trees
- Human-understandable programs
- ...

Post-hoc

- Input vs internal interpretability
- Local vs global
- Model-specific vs model-agnostic (White-box vs black-box)
- ...

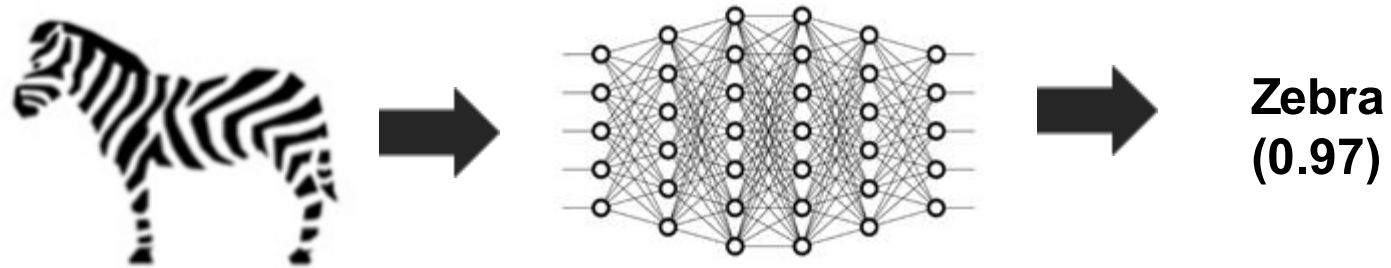
Input vs internal interpretability

- Input interpretability
 - Feature attribution
 - LIME, SHAP, Saliency Maps
 - Influence functions
 - Counterfactual explanations
 - ...
- Internal interpretability
 - Representation-based explanations
 - Mechanistic interpretability
 - ...

Representation-based explanations

- Analyzing intermediate representations of a DNN can lead to model understanding
- Concept learning: Identify concepts that are semantically meaningful to humans and the model's reliance on such concepts
- Today:
 - [Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors \(TCAV\)](#) by Kim et al., ICML'18
 - [Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task](#) by Li et al., ICLR'23

Concept-based explanations



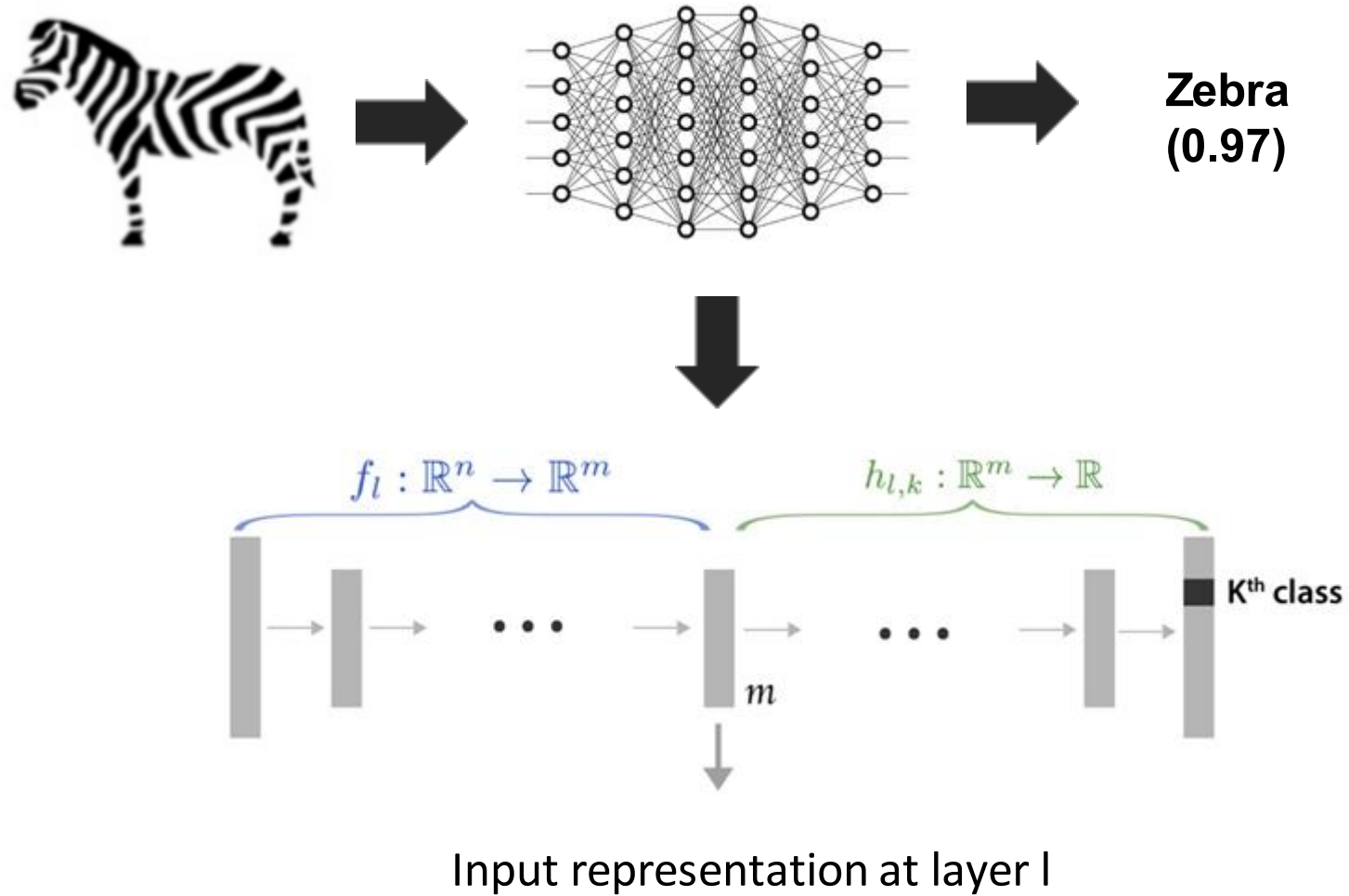
How important is the notion of “stripes” for this prediction ?

Step 1: Specifying concepts

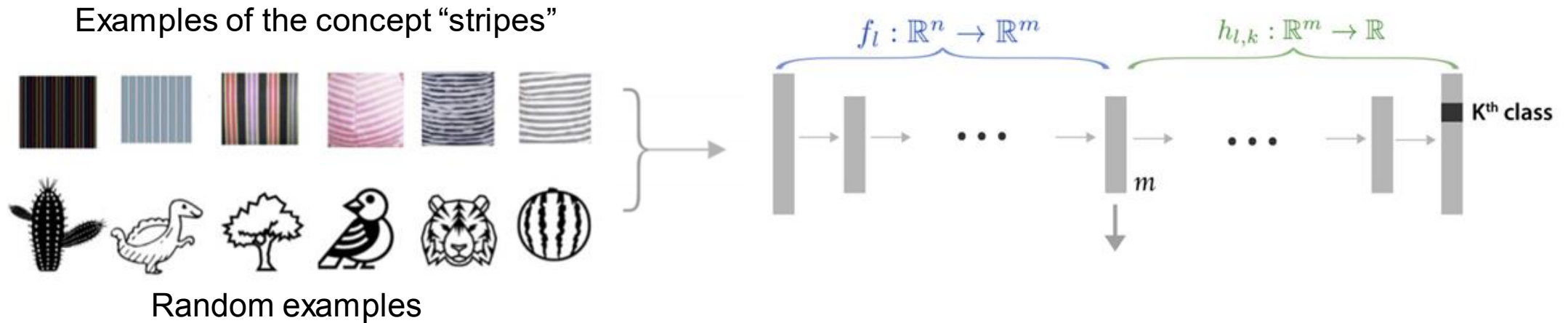
- User needs to articulate the concepts of interest
- “Stripes” can be specified by giving positive examples
- Negative examples can be chosen randomly or given by the user



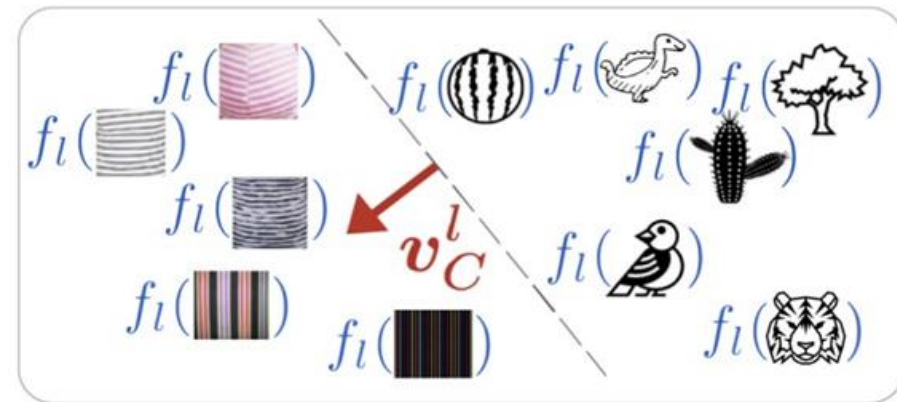
Internal layers in DNN



Step 2: Compute Concept Activation Vector (CAV)



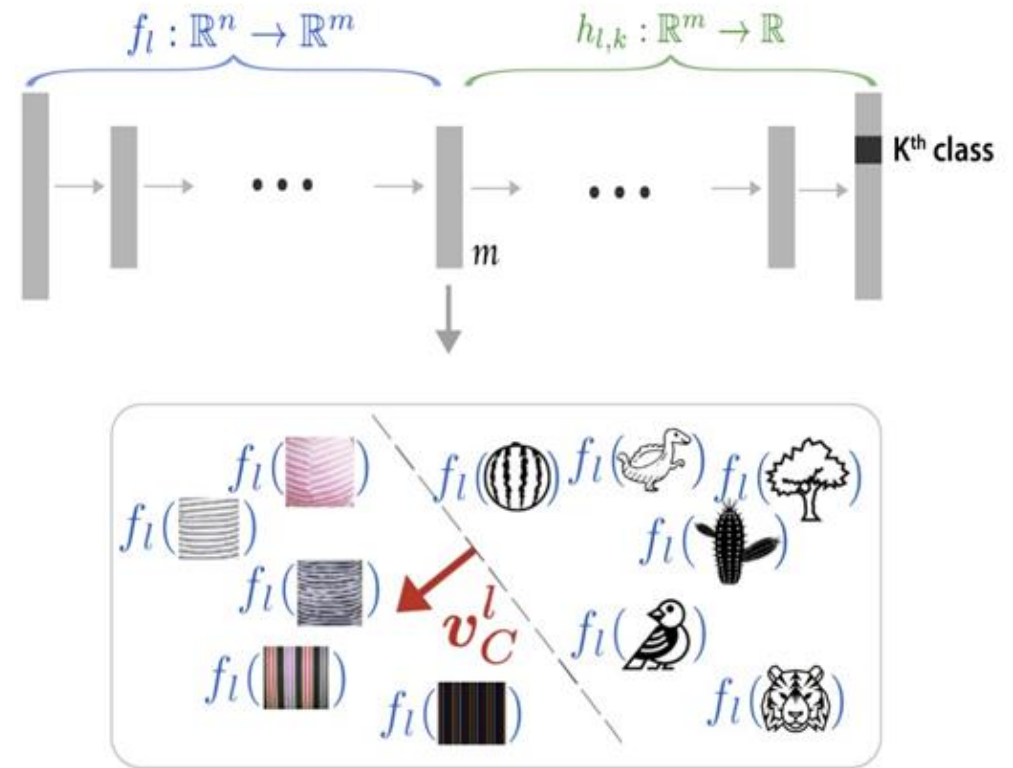
- Consider representations at layer l of all the positive and negative concept examples
- Train a linear classifier to separate positive from negative
- CAV: Vector orthogonal to the decision boundary



Step 3: Compute Conceptual Sensitivity Score

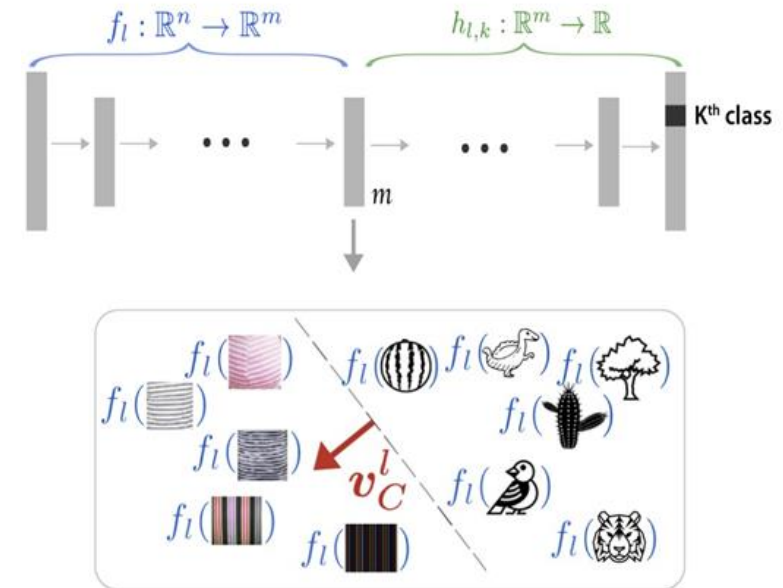
- Recall: In saliency maps, sensitivity of output to an input feature/pixel is determined by the gradient w.r.t. input x
- v_C^l is the unit CAV vector for concept C in layer l
- Conceptual sensitivity of output class k to concept C equals directional derivative:

$$\begin{aligned}
 S_{C,k,l}(x) &= \lim_{\epsilon \rightarrow 0} \frac{h_{l,k}(f_l(x) + \epsilon v_C^l) - h_{l,k}(f_l(x))}{\epsilon} \\
 &= \nabla h_{l,k}(f_l(x)) \cdot v_C^l,
 \end{aligned} \tag{1}$$



Step 4: Testing with CAV (TCAV)

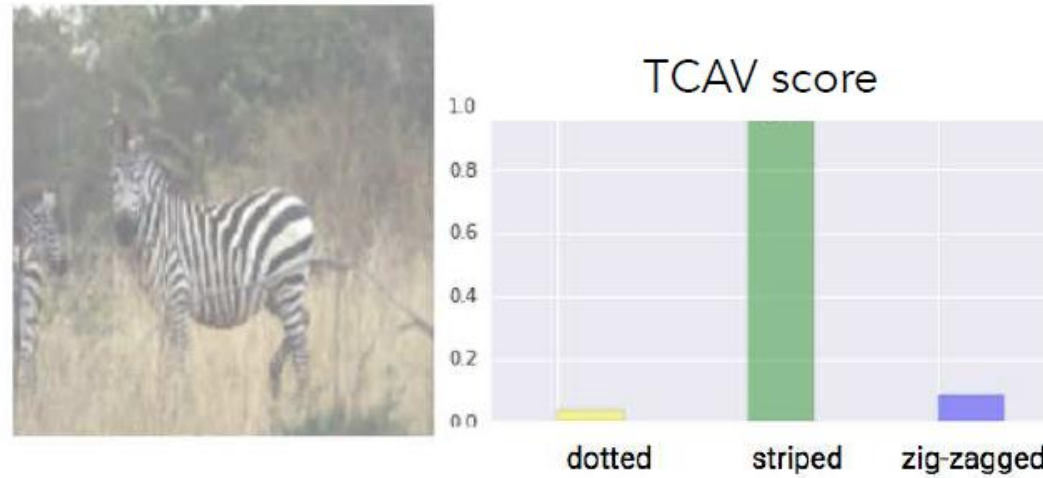
- Let X_k be the set of all training inputs with label k
- Goal: Understand how a model f 's prediction for class k is sensitive to a given concept C
- TCAV score: Fraction of k -class training inputs whose l -layer activation vector was positively influenced by concept C



$$\text{TCAV}_{Q_{C,k,l}} = \frac{|\{\mathbf{x} \in X_k : S_{C,k,l}(\mathbf{x}) > 0\}|}{|X_k|}$$

$$\begin{aligned} S_{C,k,l}(\mathbf{x}) &= \lim_{\epsilon \rightarrow 0} \frac{h_{l,k}(f_l(\mathbf{x}) + \epsilon \mathbf{v}_C^l) - h_{l,k}(f_l(\mathbf{x}))}{\epsilon} \\ &= \nabla h_{l,k}(f_l(\mathbf{x})) \cdot \mathbf{v}_C^l, \end{aligned} \quad (1)$$

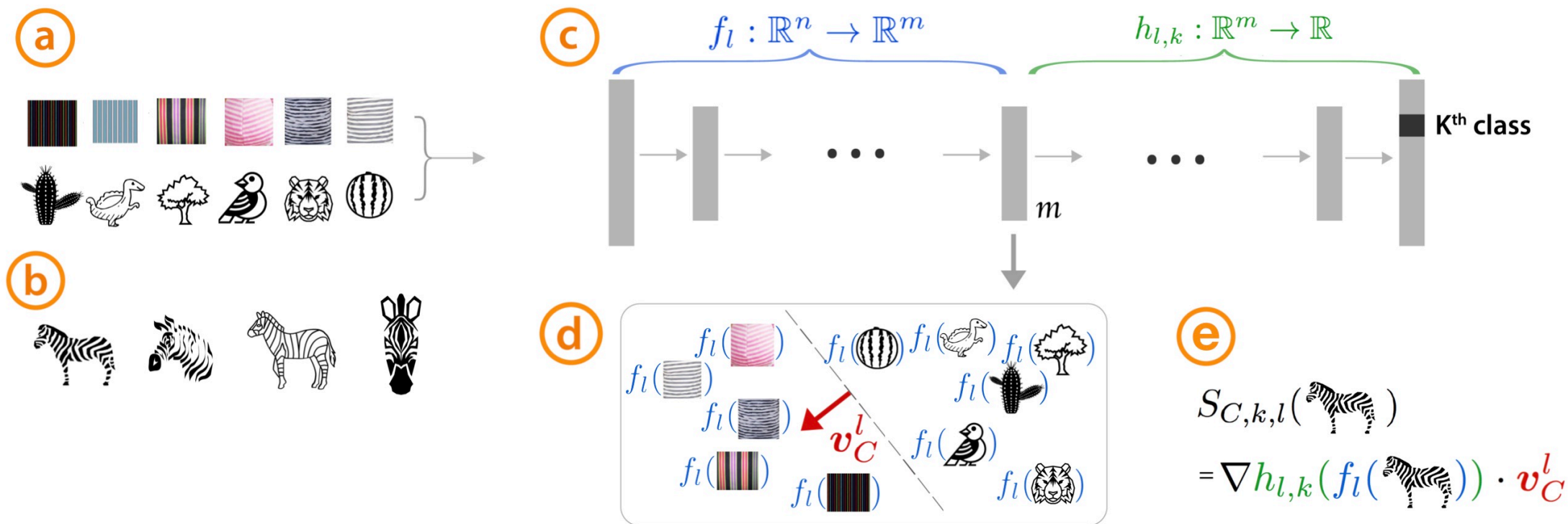
TCAV Illustration



$$\begin{aligned} \text{zebra-ness} &\rightarrow \frac{\partial h_{l,k}(z)}{\partial v_C^l} = S_{C,k,l}(\mathbf{x}) \\ \text{striped CAV} &\rightarrow \end{aligned}$$

Directional derivative with CAV

TCAV overall picture



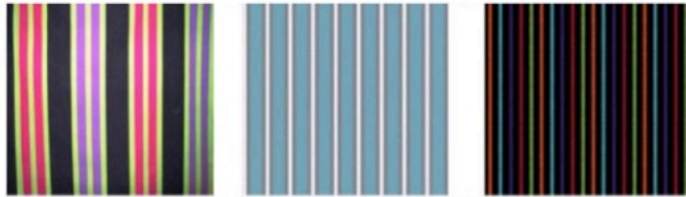
TCAV Evaluation

- (1) Sanity check experiments
- (2) Gaining insights with TCAV: biases in GoogleNet and InceptionV3
- (3) Controlled experiments with TCAV
- (4) Domain expert confirmation with Diabetic Retinopathy

Sanity check experiment (1)

- Sort images wrt a CAV using cosine similarity
- Inspect sorted images

CEO concept: most similar striped images



CEO concept: least similar striped images



Model Women concept: most similar necktie images



Model Women concept: least similar necktie images



Figure 2. The most and least similar pictures of stripes using ‘CEO’ concept (left) and neckties using ‘model women’ concept (right)

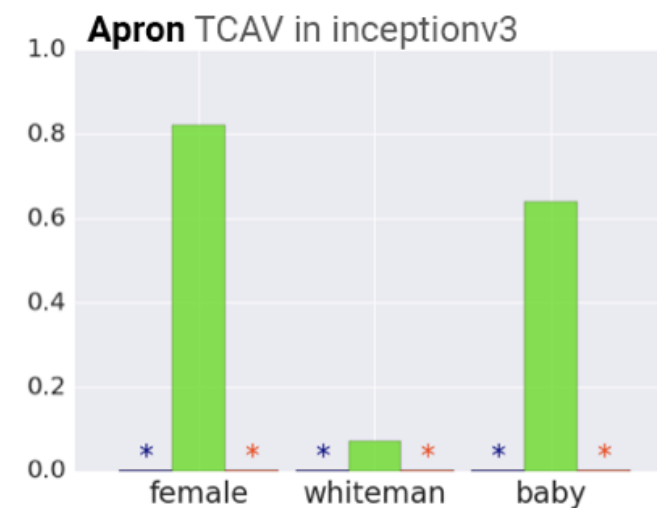
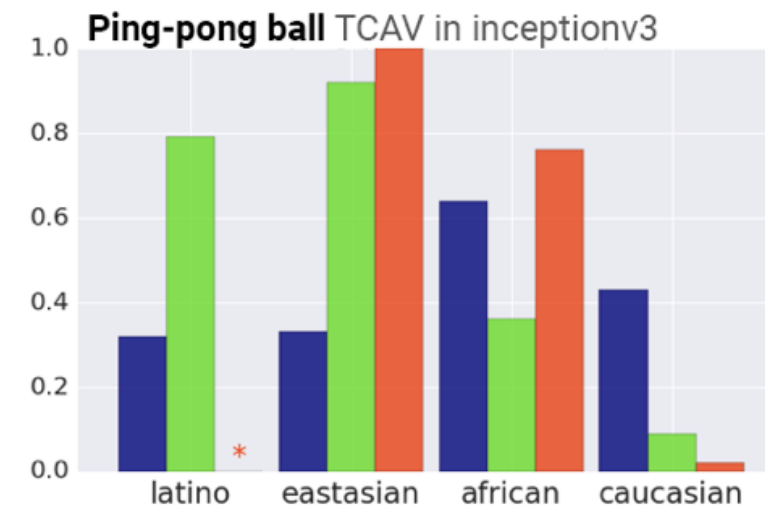
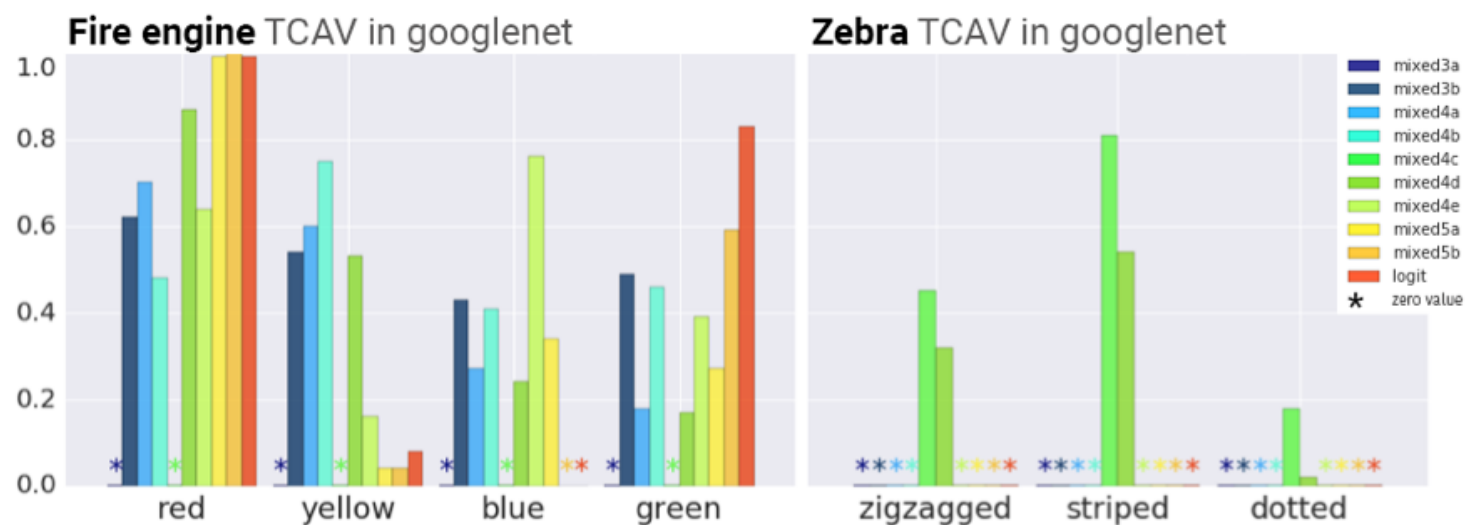
Sanity check experiment (2)

- Optimize for an input that maximally aligns with a CAV
- Start optimization from a random image



Figure 3. Empirical Deepdream using knitted texture, corgis and Siberian huskey concept vectors (zoomed-in)

Gaining insights with TCAV



Gaining insights with TCAV

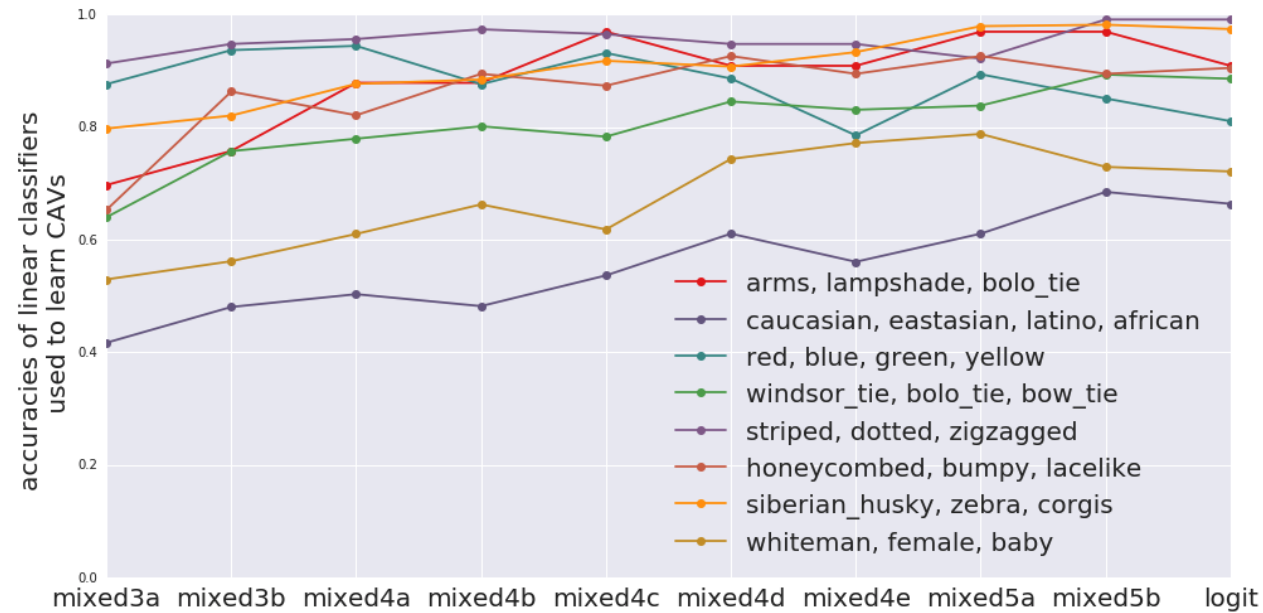
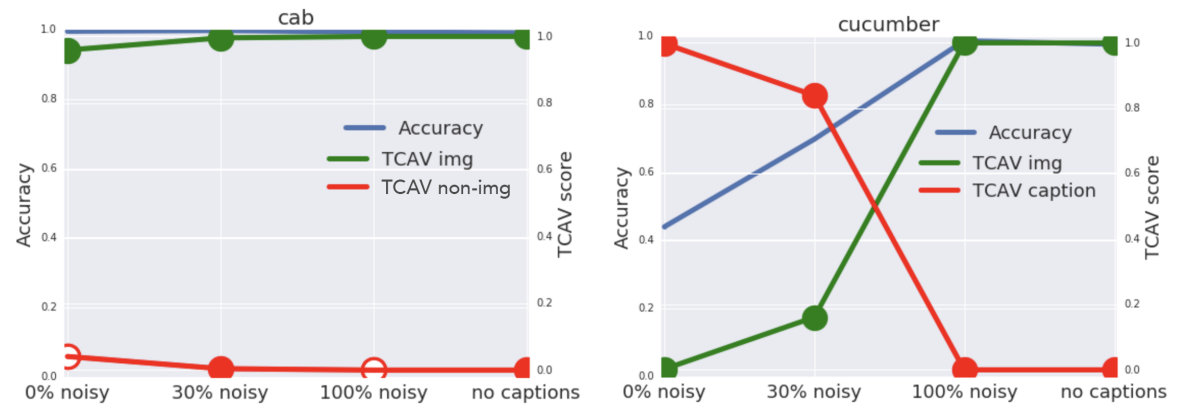
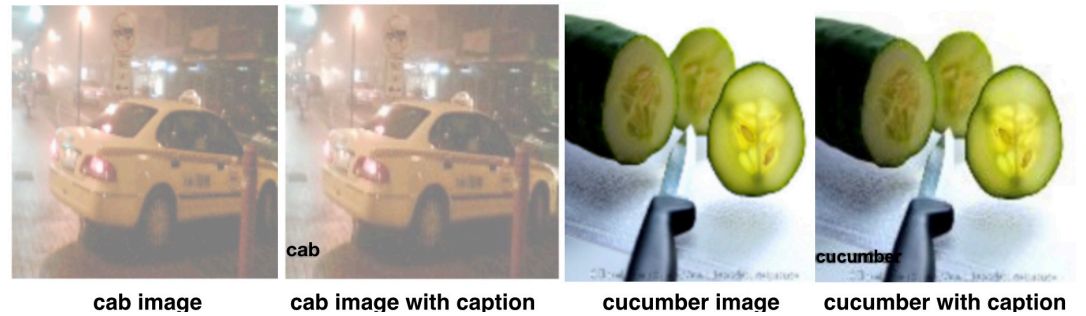


Figure 5. The accuracies of CAVs at each layer. Simple concepts (e.g., colors) achieve higher performance in lower-layers than more abstract or complex concepts (e.g. people, objects)

Controlled experiment

- Create dataset with noisy captions
- Train models on these datasets and evaluate on datasets without captions
- See TCAV scores

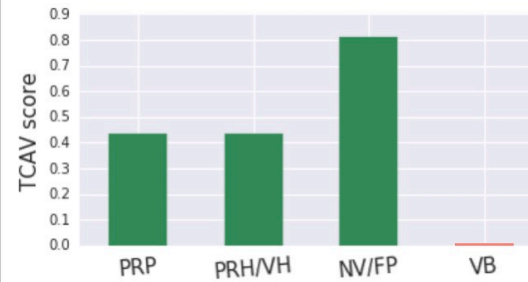


Case-study with domain experts

DR level 4 Retina



TCAV for DR level 4



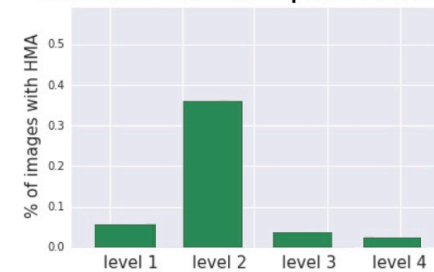
DR level 1 Retina



TCAV for DR level 1



HMA distribution on predicted DR



Potential problems with CAV

- Might not have access to data labeled with concepts
- Embeddings with and without a concept may not be linearly separable
- Concepts maybe entangled in a CAV
- See
 - Nicholson et al, [Explaining Explainability: Understanding Concept Activation Vectors](#)
 - Crabbe et al., [Concept Activation Regions: A Generalized Framework For Concept-Based Explanations](#), NeurIPS'22

Othello-GPT

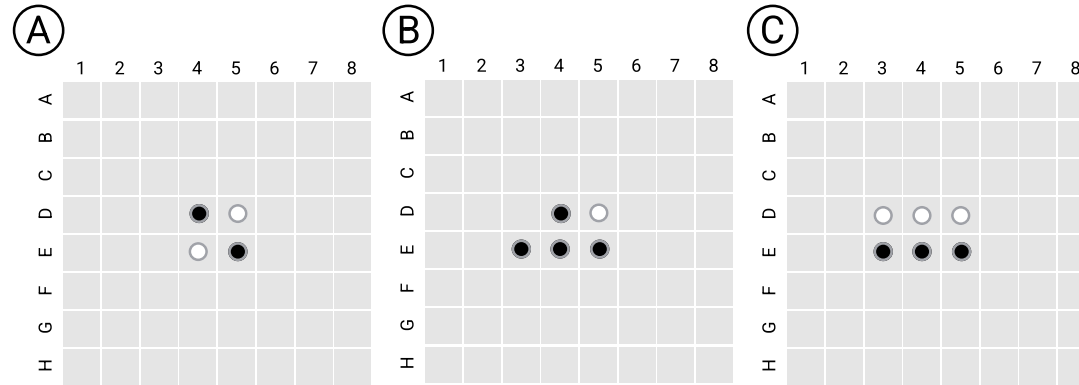


Figure 1: A visual explanation of Othello rules, from left to right: (A) The board is always initialized with four discs (two black, two white) placed in the center of the board. (B) Black always moves first. Every move must flip one or more opponent discs by outflanking—or sandwiching—the opponent disc(s). (C) The opponent repeats this process. A game ends when there are no more legal moves.

- Problem setup: Train a model to predict next legal moves for a game of Othello
- Trained Transformer model has 0.01% error rate
- Does model represent the state of the board internally?

Probing internal representations

- For each tile of the board, train a 3-way classifier to predict if tile is Empty, White, or Black
- High-error rates for linear classifiers:

	x^1	x^2	x^3	x^4	x^5	x^6	x^7	x^8
Randomized	26.7	27.1	27.6	28.0	28.3	28.5	28.7	28.9
Championship	24.2	23.8	23.7	23.6	23.6	23.7	23.8	24.3
Synthetic	21.9	20.5	20.4	20.6	21.1	21.6	22.2	23.1

Table 1: Error rates (%) of linear probes on randomized Othello-GPT and Othello-GPTs trained on different datasets across different layers (x^i represents internal representations after the i -th layer).

- Low-error rates for non-linear classifiers:

	x^1	x^2	x^3	x^4	x^5	x^6	x^7	x^8
Randomized	25.5	25.4	25.5	25.8	26.0	26.2	26.2	26.4
Championship	12.8	10.3	9.5	9.4	9.8	10.5	11.4	12.4
Synthetic	11.3	7.5	4.8	3.4	2.4	1.8	1.7	4.6

Table 2: Error rates (%) of nonlinear probes on randomized Othello-GPT and Othello-GPTs trained on different datasets across different layers. Standard deviations are reported in Appendix H.

Validating probes with interventional experiments

- Use the probes to change the internal activations of the model
- Check if the next predicted move reflects the updated board state

